Animal Shelter Management System

A C# Console Application for Animal Care

Moamen Mohamed & Ester Emad

May 3rd, 2026

Who Uses It & Why?

Local shelters rely on scattered paperwork, disjointed spreadsheets, and manual memory to track intake, medical status, and adoptions. The result: lost records, missed vaccinations, and missed matches between animals and adopters.

📋 Lost Records

Animal histories disappear between sessions and staff changes.

💉 Missed Care

Vaccinations and treatments slip through the cracks without structured tracking.

🔍 Inefficient Matching

Adopters and animals are harder to connect without searchable, organized data.

The Solution

A terminal-based C# console application that gives shelters a structured, persistent way to manage every animal in their care – no database or internet required.

4 Animal Types

Dogs, Cats, Birds, and Small Animals – each with unique fields like breed, habitat, and wingspan.

Full Care Tracking

Adoption status, care notes, and vaccine records all tied to each animal's profile.

Persistent Data

All records save automatically to a .txt file – nothing is lost between sessions.

Zero Dependencies

Runs entirely in the terminal. No database, no internet, no complex setup.

Application Menu

Six core features cover the full lifecycle of shelter animal management.

1 **Add Animal**
Choose a type and enter breed, age, status, and care details.

2 **View All**
Display every animal in a clean, formatted list.

3 **Search**
Find animals by name, ID, or adoption status.

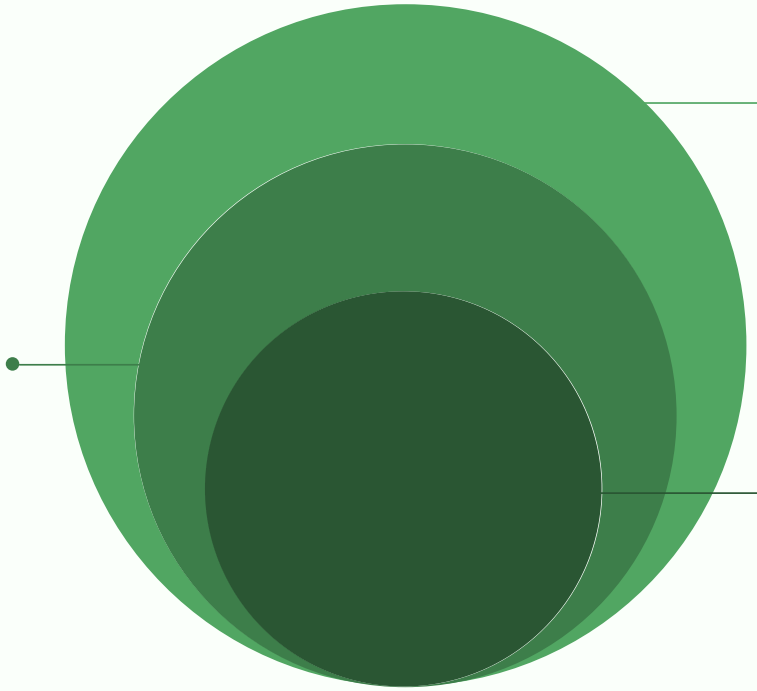
4 **Update Status**
Mark as Adopted, Under Medical Care, Pending, and more.

5 **Remove Animal**
Delete with a confirmation prompt to prevent accidents.

6 **Auto-Save on Exit**
Data persists to a .txt file automatically every session.

How It All Fits Together

The system follows a clean layered architecture – from user input through business logic to persistent storage – keeping each concern separate and testable.



Controller

Receives: IAnimalRepository (from Program)

Creates: AnimalService(repository)

Depends on: IAnimalService

Service & Factory

Receives: IAnimalRepository

For Add: calls AnimalFactory to build the object, then repo.Add()

For Adopt: calls animal.Adopt() (IAdoptable)

Repo

AnimalManager (IAnimalRepository)

Owns: List<Animal>

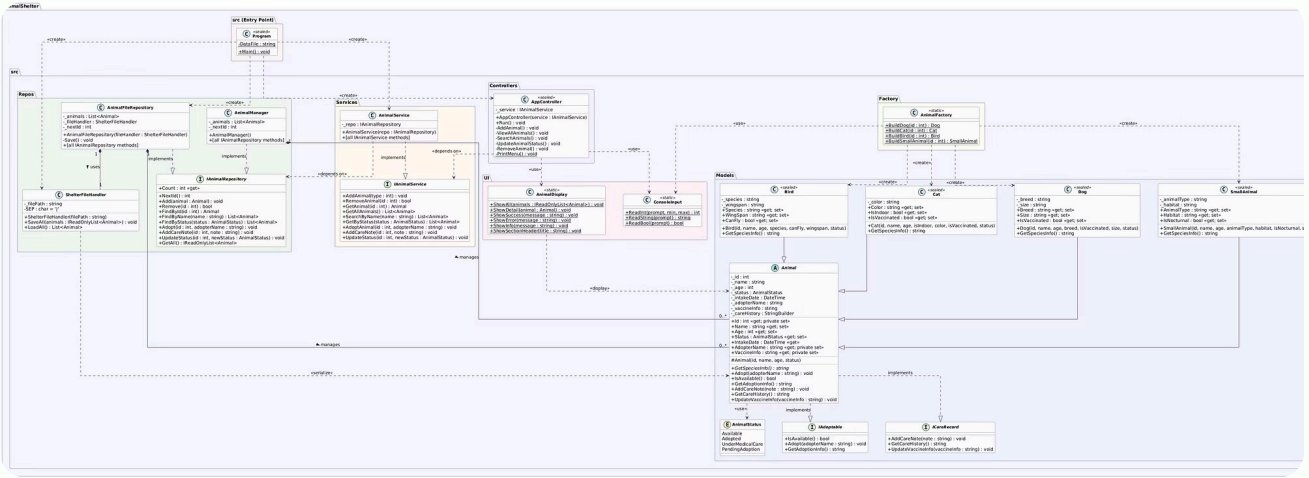
ShelterFileHandler for File I/O

The **Program.cs** creates:

ShelterFileHandler → AnimalFileRepository → AnimalService → AppController

Class Diagram

Eight responsibility groups form the class structure – from abstract domain models to the Program entry point.



Layered Responsibilities

Each group has a single, clear role – making the system easy to extend, test, and maintain.

- **Models:** hold domain data and behavior
- **Interfaces:** define adoption and care contracts
- **Repos:** manage in-memory storage and file I/O
- **Services:** business logic – what you can do with animals
- **Factory:** object creation – builds animals from user input
- **Controllers:** menu flow – coordinates everything
- **UI + Program:** handle input, display, and wiring

Polymorphism & Abstraction

One Class, Many Forms

The abstract `Animal` base class defines `GetSpeciesInfo()`. Each subclass `Dog`, `Cat`, `Bird`, `SmallAnimal` overrides it with its own details. The UI calls this method on any animal without knowing its concrete type.

```
// Base Class
public abstract string GetSpeciesInfo();

// Subclass (Dog) implementation
public override string GetSpeciesInfo()
 => $"Dog | Breed: {Breed} | Size: {Size}";
```

```
// Calls GetSpeciesInfo() on every animal
// without knowing its concrete type

foreach (var animal in animals)
  Console.WriteLine(animal.GetSpeciesInfo());

// Output varies: "Dog | ...", "Cat | ...", "Bird | ..."
```

Encapsulation & Interface

Protecting Business Rules

Private fields, validated public properties, `private set` on sensitive properties

```
private int _id;
```

```
public int Id
{
  get => _id;
  private set
  {
    if (value <= 0)
      throw new ArgumentException("Id must be
greater than zero.", nameof(value));
    _id = value;
  }
}
```

Focused contract

define adoption and care contracts

```
public interface IAdoptable
{
   bool   IsAvailable();
   void   Adopt(string adopterName);
   string GetAdoptionInfo();
}
```

SOLID Design Principles

S — Single Responsibility

Each class has one job. AnimalDisplay only displays. AnimalManager only stores.

O — Open/Closed

AnimalFactory adds a new animal type by creating a subclass – nothing else changes.

L — Liskov Substitution

Dog, Cat, Bird, and SmallAnimal substitute for Animal anywhere.

I — Interface Segregation

IAdoptable and ICareRecord are separate – classes implement only what they need.

D — Dependency Inversion

AppController depends on IAnimalService, not a concrete class. Program injects the implementation once.

Video Demo

Watch the Animal Shelter Management System in action – from adding animals and updating their status to searching records and persisting data across sessions.

Google Docs

Animal Shelter Management System Video Demo.mp4